

MapReduce: Exercises

From the Slides 62 and 63 on MapReduce

Exercise

For each of the 4 problems listed in Exercise 2.3.1 of [LRU14], design an $O(1)$ -round MR algorithm which uses $O(\sqrt{N})$ local space and linear aggregate space, where N is the input size.

Solution to 2.3.1 (b) of [LRU14]

Problem: given a set S of N integers $x_0, x_1, \dots, x_{N-1}$ compute their *arithmetic mean*.

$$\text{mean}(S) = \frac{1}{N} \sum_{i=0}^{N-1} x_i.$$

Assume x_i represented by pair (i, x_i) .

- **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
- **Output:** $(0, \text{mean}(S))$.

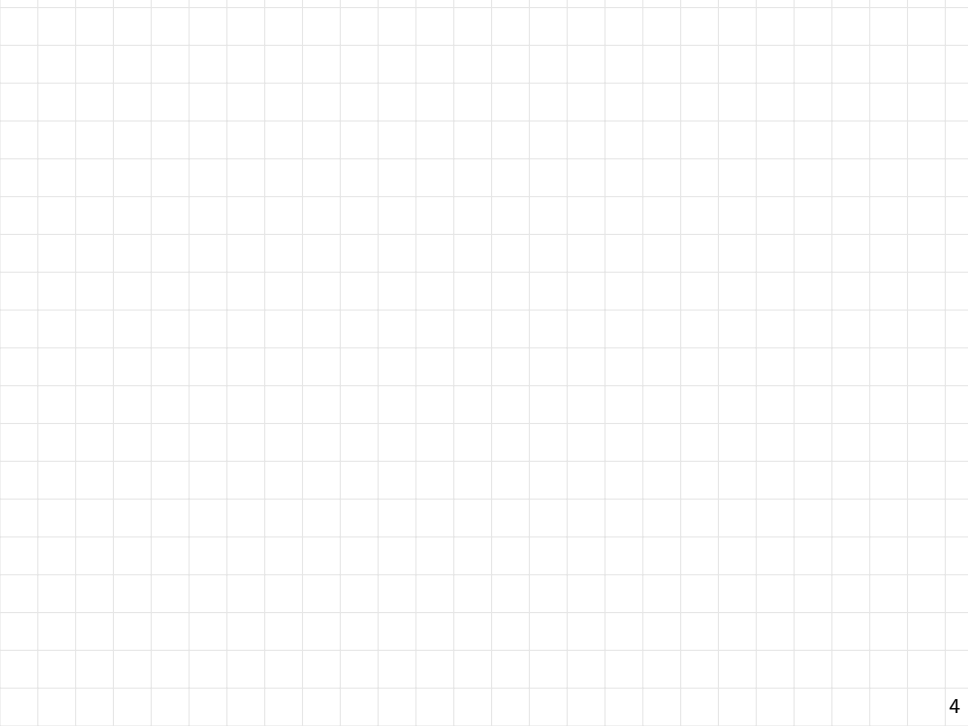
Goals: $R = O(1)$, $M_L = O(\sqrt{N})$, $M_A = O(N)$.

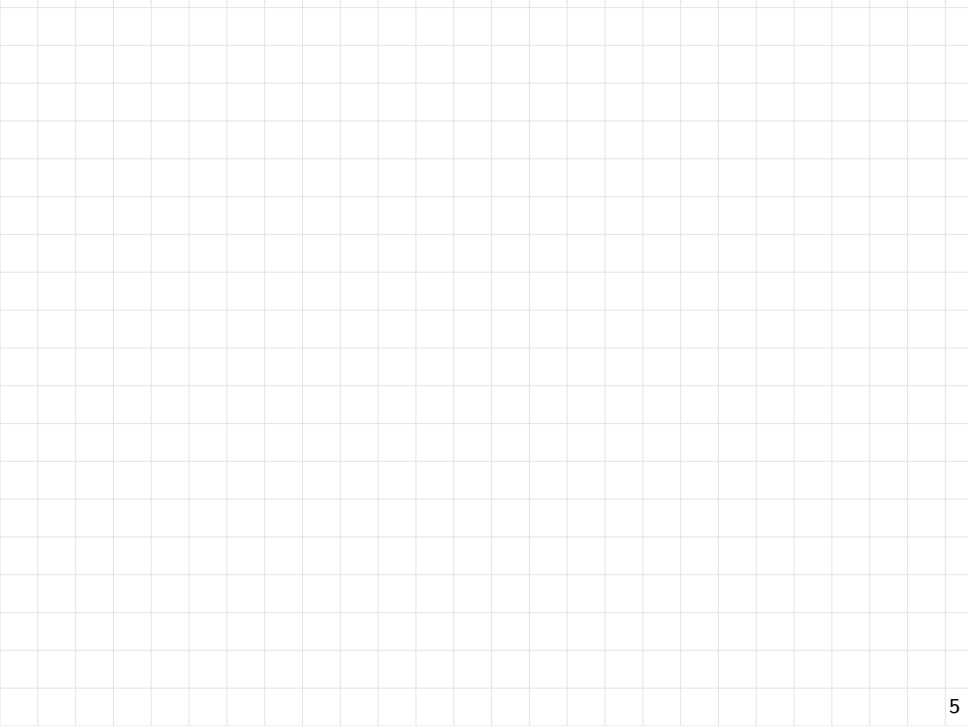
IDEA

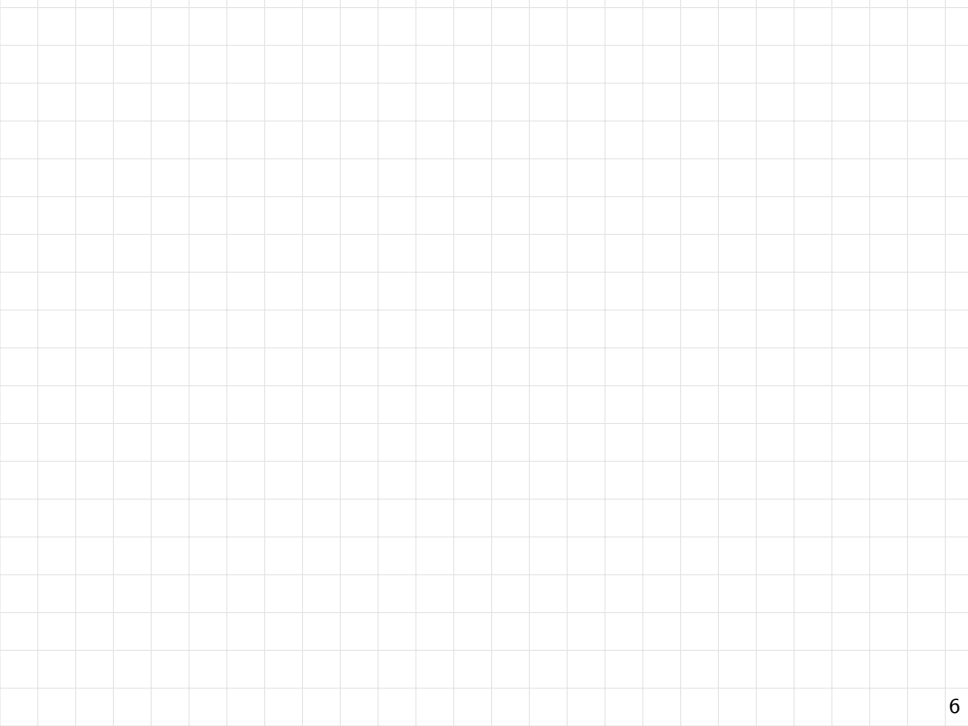
Round 1: Subdivide S into $\sqrt{N}$ partitions
of size $\sqrt{N}$ each and compute
partial sums in each partition

Round 2: Aggregate the $\sqrt{N}$ partial
sums

Use N as global variable







Solution to 2.3.1 (d) of [LRU14]

Problem: given a set S of N integers $x_0, x_1, \dots, x_{N-1}$, possibly repeated, compute the *number of distinct integers* occurring in S

Example: $\{9, 7, 3, 7, 7, 9, 15\} \Rightarrow 4$ distinct integers.

Goals: $R = O(1)$, $M_L = O(\sqrt{N})$, $M_A = O(N)$.

We'll see 2 algorithms:

- **Algorithm 1:** straightforward, $R = 2$, possibly large M_L
- **Algorithm 2:** meets goals ($R = 4$)

Input-output specification:

- **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
- **Output:** $(0, \text{count})$, where $\text{count} = \#$ distinct integers in S .

Algorithm 1. Idea

R1: Eliminate all duplicates

R2: Count distinct elements

Round 1: Eliminate duplicates

Map Phase $(i, x_i) \rightarrow (x_i, i)$

key

Reduce Phase $\forall$ integer x gather all pairs $(x = x_i, i)$ into (x, L_x) L_x list of indices i such that $x_i = x$

return $(0, x)$

Round 2 · count distinct elements

Map Phase : empty

Reduce Phase · Gather all pairs $(0, 2)$
into $(0, L)$ L list of distinct integers
return $(0, |L|)$

Analysis

2 Rounds

$M_A = O(N)$ no replication but only
aggregation and elimination of data

M_L

* N_1 = max # of copies of an input int

* N_2 = # distinct integers in S

Round 1

Map. $\Theta(1)$

Reduce. $\Theta(N_1)$

Round 2

Map. -

Reduce. $\Theta(N_2)$

$$\Rightarrow M_L = \Theta(\max\{N_1, N_2\})$$

Goals: $R = O(1)$ $M_L = O(\sqrt{N})$ $M_A = O(N)$

Both N_1 and N_2 can be as large as $\Theta(N)$

$$N_1 \cdot N_2 \geq N$$

$$\max\{N_1, N_2\} \geq \sqrt{N}$$

Algorithm 2

* Duplicate removal 2 rounds

* Count 2 rounds

$\Rightarrow$ 4 rounds

Round 1

Map Phase: $(i, x_i) \rightarrow (i \bmod \sqrt{N}, x_i)$

Reduce Phase: $\forall j \in [0, \sqrt{N})$ create (j, L_j)

$L_j \equiv$ list of integers x from pairs (j, x)

return ONE pair (j, x) for each distinct

$x \in L_j$

AT THIS POINT: $\leq \sqrt{N}$ copies for each integer

Round 2

Map Phase $(j, x) \rightarrow (x, j)$

Reduce Phase For each integer x create (x, L_x) L_x list of indices j such that there is a pair (x, j)

return ONE (x, j) for an arbitrary $j \in L_x$

AT THIS POINT

* Only one copy left for every integer

* $\leq \sqrt{N}$ pairs with value j

Round 3

Map Phase $(x, j) \rightarrow (j, x)$

Reduce Phase $\forall j \in [0, \sqrt{N})$ create (j, L_j)

$L_j \equiv$ list of integers from pairs (j, x)

return $(0, |L_j| = c_j)$

Round 4

Map Phase : empty

Reduce Phase : gather all pairs $(0, c_j)$

$j \in [0, \sqrt{N})$ into $(0, L)$ $L \equiv$ list of c_j 's

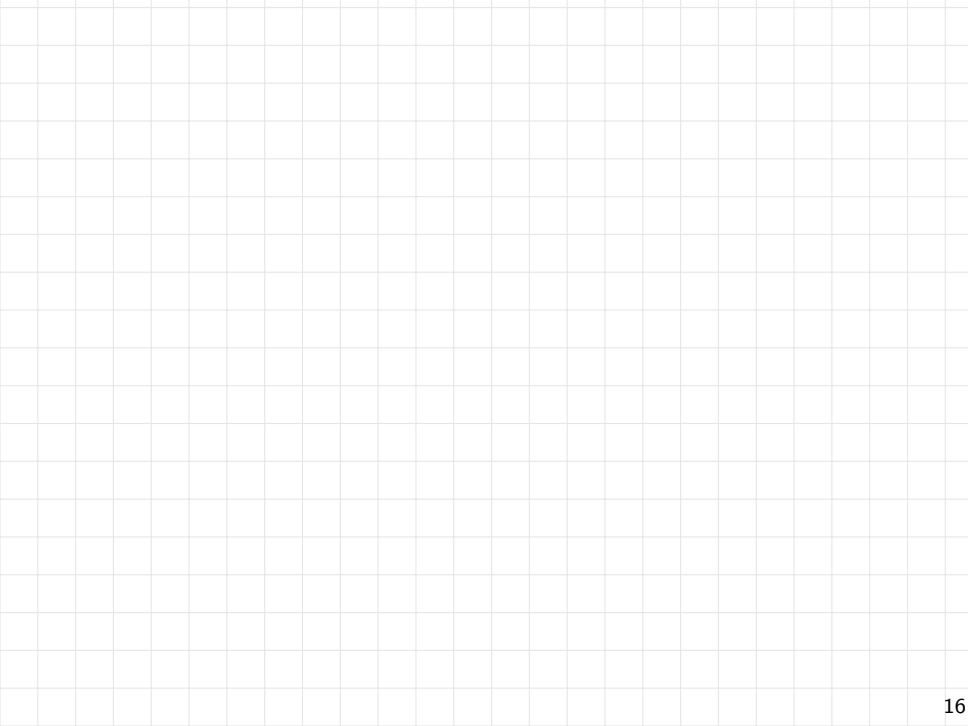
return $(0, \sum_{c_j \in L} c_j)$

Analysis

4 Rounds

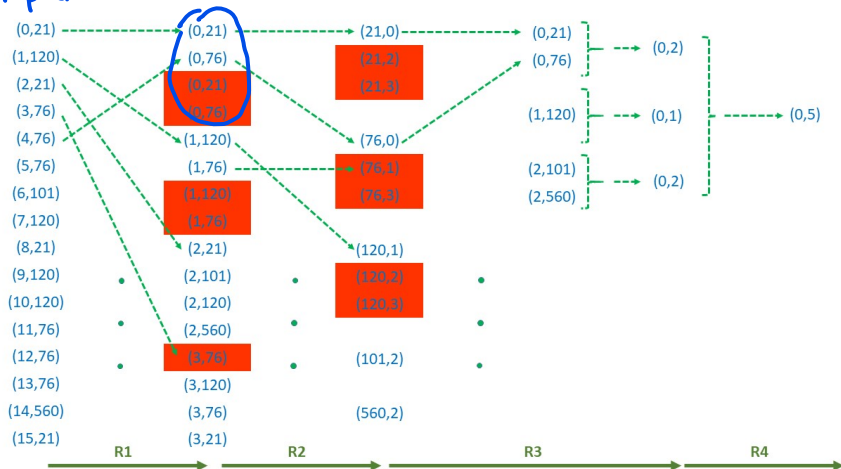
$M_A = O(N)$ no replication but only aggregation and elimination of data

$M_L = O(\sqrt{N})$ because of the initial partitioning and of the fact that after Round 1 $\leq \sqrt{N}$ copies of each integer survived



Example ($N = 16$)

Input $N=16$



Exercise

Design and analyze an efficient MR algorithm for approximating the maximum pairwise distance, which uses local space $M_L = O(N^{1/4})$. Can you generalize the algorithm to attain $M_L = O(N^\epsilon)$, for any $\epsilon \in (0, 1/2)$?

Solution with $M_L = O(N^{1/4})$

Input-output specification:

- **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
- **Output:** $(0, d_{\max}(0) = \max_{0 \leq i < N} d(x_0, x_i))$.

Obs: by theory, $\max \text{ pairwise distance} \in [d_{\max}(0), 2d_{\max}(0)]$.

Let's assume x_0 **as a global variable.**

Goals: $R = O(1)$, $M_L = O(N^{1/4})$, $M_A = O(N)$.

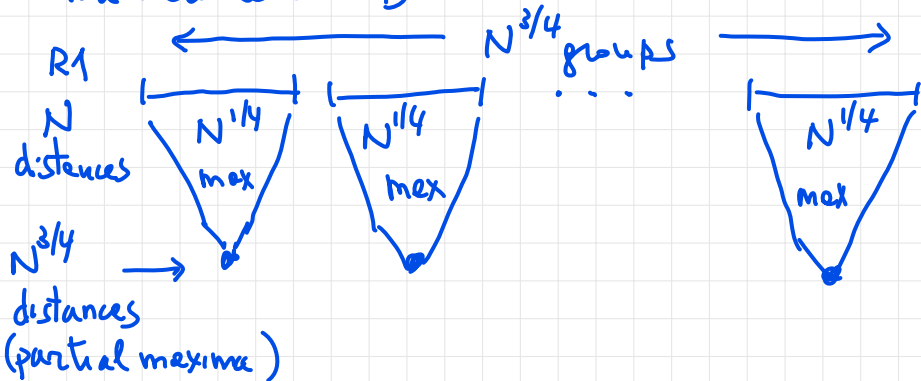
and distance function
known to the
program

IDEA (fill in details as an exercise)

A. Compute all distances $d(x_0, x_i)$
 $\forall i \in [0, N)$. Map Phase of Round 1
 $\Rightarrow M_L = O(1)$

B. Compute $\max \{d(x_0, x_i) \mid 0 \leq i < N\}$
aggregating groups of $N^{1/4}$ distances
using suitable keys to define groups

More details on B



R2 $N^{3/4}$ distances $\rightarrow$ $N^{1/2}$ distances
 using $N^{1/2}$ groups of size $N^{1/4}$ each
 $\Rightarrow$ end up with $N^{1/2}$ partial maxima

R3 $N^{1/2}$ distances $\rightarrow N^{1/4}$ distances
using $N^{1/4}$ groups of size $N^{1/4}$ each
 $\Rightarrow$ end up with $N^{1/4}$ partial max use

R4 Compute the final maximum ($d_{\max}(o)$)

Analysis

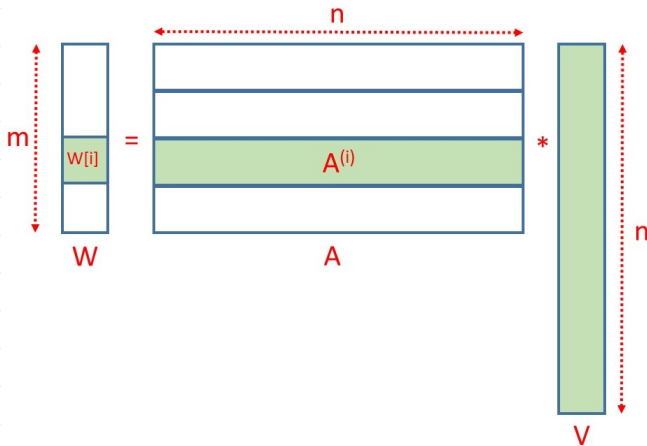
Rounds: 4

$M_A = O(N)$ no replication but only aggregation
 $M_L = O(N^{1/4})$ in each round process groups of
size $N^{1/4}$, individually

Exercise

Design an $O(1)$ -round MR algorithm for computing a matrix-vector product $W = A \cdot V$, where A is an $m \times n$ matrix, V is an n -vector, and $m \leq \sqrt{n}$. Your algorithm must use $o(n)$ local space and linear (i.e., $O(mn)$) aggregate space.

How would your solution change if m were larger?



Input

$$* A = \{((i, j), A[i, j]) \mid i \in [0, m), j \in [0, n)\}$$

key value

$$* V = \{((j, -1), V[j]) \mid j \in [0, n)\}$$

key value

$$\text{Output} \cdot W = \{((i, W[i]) \mid i \in [0, m)\}$$

key value

Notation $A^{(i)} \equiv \text{row } i \text{ of } A \quad i \in [0, m)$

1-Round Algorithm

Map Phase

$$((i, j), A[i, j]) \rightarrow (i, \overbrace{((i, j), A[i, j])}^{\text{value}})$$

$$((j, -1), V[j]) \rightarrow \{(i, ((j, -1), V[j])) \cdot i \in [0, m)\}$$

$\Rightarrow m$ copies of V !

Notation . $V^{(i)} \equiv \text{copy } i \text{ of } V \quad i \in [0, m)$

$$\text{Obs } W[i] = A^{(i)} \cdot V^{(i)}$$

$\searrow$ inner product

Reduce Phase $\forall i \in [0, m)$ gather all pairs with
key i into (i, L_i) where L_i is the list of
pairs representing entries of $A^{(i)}$ and $V^{(i)}$
RETURN $W[i] = A^{(i)} \cdot V^{(i)}$

Analysis

$$M_A = O(|A| + m|V|) = O(m \cdot n)$$

$\Rightarrow$ linear in the input size

$$M_L = O(m + n)$$

Map

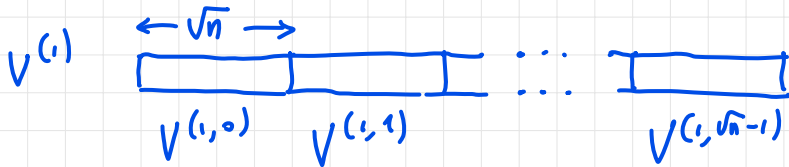
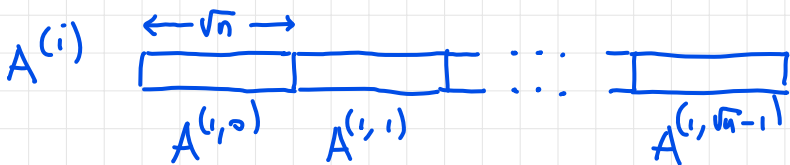
Reduce

NOT $O(n)$!

but note that $m \leq \sqrt{n}$

How can we improve M_L ?

IDEA: Compute $A^{(i)} \cdot V^{(i)}$ in blocks of $\sqrt{n}$ entries each



$$|A^{(i,s)}| = |V^{(i,s)}| = \sqrt{n} \quad \forall s \in [0, \sqrt{n})$$

Define $W_s[i] = A^{(i,s)} \cdot V^{(i,s)}$

$$\Rightarrow W[i] = \sum_{s=0}^{\sqrt{n}-1} W_s[i]$$

Round 1

Map Phase

- * replicate each entry of V m times
- * assign keys to create a group for each (i,s) with the entries of $A^{(i,s)}$ and $V^{(i,s)}$

Reduce phase: compute $W_s[i] \quad \forall (i,s)$

Round 2

Map Phase: empty

Reduce Phase: Compute $W[i] = \sum_{s=0}^{\sqrt{n}-1} W_s[i]$

and return $W[i] \quad \forall i \in [0, m)$

ANALYSIS

$M_A = O(m \cdot n)$ as before

$M_L = O(m + \sqrt{n}) = O(\sqrt{n})$ recall $m \leq \sqrt{n}$

copies
of V

computation of each
 $W_s[i]$ and of $W[i]$

Exercise . fill in details

